

1. Préparation de l'environnement de travail

Le terminal Windows PowerShell ne dispose pas nativement des commandes Linux (mkdir avec accolades, tree, tar, apt, etc.). J'ai donc activé **WSL2** avec une distribution **Ubuntu 24.04**, puis mis à jour la liste des paquets disponibles avant d'installer l'utilitaire **tree**, qui permet d'afficher l'arborescence d'un dossier de façon lisible.

■ WSL — Ubuntu 24.04

```
root@DESKTOP-PTBFF3I:~/IA_Project# wsl -l -v
NAME STATE VERSION
* docker-desktop Running 2
Ubuntu Stopped 2
root@DESKTOP-PTBFF3I:~/IA_Project# wsl -d Ubuntu
root@DESKTOP-PTBFF3I:~/IA_Project# apt update
Hit/Get ... noble-security, noble, noble-updates, noble-backports
Fetched 5181 kB in 7s
Reading package lists... Done
2 packages can be upgraded.
root@DESKTOP-PTBFF3I:~/IA_Project# apt install -y tree
tree is already the newest version (2.1.1-2ubuntu3.24.04.2).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
```

Résultat / interprétation : Le sous-système Linux Ubuntu est fonctionnel, la liste des paquets est à jour et l'outil `tree` est disponible pour la suite de l'atelier.

2. Partie 1 — Création de l'arborescence du projet

La commande `mkdir -p` avec des accolades permet de créer en une seule instruction toute une arborescence de dossiers imbriqués. Elle a été utilisée pour structurer le projet **IA_Project** avec un dossier par type de contenu : données brutes et nettoyées, configuration, journaux (logs), scripts, modèles, API, sauvegardes, documentation et un dossier partagé.

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# mkdir -p
~/IA_Project/{datasets/brut,datasets/clean,config,logs,scripts,models,api,backu
root@DESKTOP-PTBFF3I:~/IA_Project# p,documentation,shared}
root@DESKTOP-PTBFF3I:~/IA_Project# cd ~/IA_Project
root@DESKTOP-PTBFF3I:~/IA_Project# tree .
```

```
root@DESKTOP-PTBFF3I: ~/IA_Project
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
root@DESKTOP-PTBFF3I:/mnt/c/Users/dell# mkdir -p ~/IA_Project/{datasets/brut,datasets/clean,config,logs,scripts,models,api,backup,documentation,shared}
root@DESKTOP-PTBFF3I:/mnt/c/Users/dell# cd ~/IA_Project
e .root@DESKTOP-PTBFF3I:~/IA_Project# tree .
.
├── api
├── backup
├── config
│   ├── model.conf
│   └── settings.conf
├── datasets
│   ├── brut
│   │   ├── clients.csv
│   │   ├── readme.txt
│   │   └── train.csv
│   └── clean
│       └── train.csv
├── documentation
├── logs
│   ├── application.log
│   ├── full.log
│   ├── loss.log
│   └── training.log
├── models
│   └── modele.txt
├── scripts
│   ├── preprocess.py
│   └── train.py
└── shared

12 directories, 13 files
root@DESKTOP-PTBFF3I:~/IA_Project#
```

Capture d'écran réelle — résultat de la commande « tree . » sur l'arborescence créée.

Résultat / interprétation : La commande `tree .` confirme la création de **12 dossiers et 13 fichiers** (fichiers présents par défaut au moment de la capture : `config/model.conf`, `config/settings.conf`, les CSV de `datasets`, les logs, `models/modèle.txt` et les scripts `preprocess.py` / `train.py`). La structure respecte l'organisation demandée : les données brutes ne doivent jamais être modifiées directement, seules leurs versions nettoyées vont dans `datasets/clean`.

3. Partie 2 — Exploration du contenu des fichiers

La commande `cat` affiche l'intégralité du contenu d'un fichier texte. Elle a été utilisée pour lire le fichier explicatif du dossier de données brutes, le journal des pertes d'entraînement, et la fiche descriptive du modèle.

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# cat datasets/brut/readme.txt
Projet : IA de prédiction des achats
```

```
Ce dossier contient les données brutes destinées
à l'entraînement d'un modèle de Machine Learning.
```

```
Fichiers disponibles : train.csv, test.csv, clients.csv
Ne jamais modifier directement les fichiers originaux.
Les versions nettoyées devront être placées dans datasets/clean.
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# cat logs/loss.log
```

```
Epoch 1 : Loss = 0.82
```

```
Epoch 2 : Loss = 0.60
```

```
Epoch 3 : Loss = 0.45
```

```
Epoch 4 : Loss = 0.32
```

```
Epoch 5 : Loss = 0.28
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# cat models/modèle.txt
```

```
Nom du modèle : Random Forest
```

```
Version : 1.0
```

```
Date : 18/07/2026
```

```
Accuracy : 94 %
```

```
Auteur : Equipe IA
```

Résultat / interprétation : Le `readme` confirme la règle de non-modification des fichiers bruts. Le journal des pertes montre une **baisse régulière de la Loss** (0.82 → 0.28) au fil des 5 epochs, signe que l'entraînement converge correctement. La fiche modèle indique un Random Forest avec **94 %**

d'accuracy.

4. Partie 3 — Manipulation et inspection de fichiers

Plusieurs commandes complémentaires permettent d'inspecter rapidement un fichier sans l'ouvrir en entier : `head` (premières lignes), `tail` (dernières lignes), `wc -l` (nombre de lignes) et `grep` (recherche de motif).

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# head -5 datasets/brut/train.csv
id,age,revenu,ville,achat
1,25,350000,Dakar,Oui
2,42,720000,Thies,Non
3,31,500000,Saint-Louis,Oui
4,28,410000,Dakar,Oui
root@DESKTOP-PTBFF3I:~/IA_Project# tail -3 logs/application.log
2026-07-18 08:00:17 WARNING Valeurs manquantes détectées
2026-07-18 08:00:20 INFO Nettoyage terminé
2026-07-18 08:00:30 INFO Modèle prêt
root@DESKTOP-PTBFF3I:~/IA_Project# wc -l datasets/brut/clients.csv
6 datasets/brut/clients.csv
root@DESKTOP-PTBFF3I:~/IA_Project# grep "WARNING" logs/application.log
2026-07-18 08:00:17 WARNING Valeurs manquantes détectées
root@DESKTOP-PTBFF3I:~/IA_Project# grep -r "INFO" logs/
logs/application.log:2026-07-18 08:00:10 INFO Application démarrée
logs/application.log:2026-07-18 08:00:15 INFO Chargement du dataset
logs/application.log:2026-07-18 08:00:20 INFO Nettoyage terminé
logs/application.log:2026-07-18 08:00:30 INFO Modèle prêt
logs/full.log:2026-07-18 08:00:10 INFO Application démarrée
logs/full.log:2026-07-18 08:00:15 INFO Chargement du dataset
logs/full.log:2026-07-18 08:00:20 INFO Nettoyage terminé
logs/full.log:2026-07-18 08:00:30 INFO Modèle prêt
```

Résultat / interprétation : Le fichier `train.csv` contient bien des colonnes `id/age/revenu/ville/achat`. Le journal applicatif se termine sur « Modèle prêt ». Le fichier `clients.csv` compte 6 lignes. Une seule ligne `WARNING` est détectée (valeurs manquantes), et le mot-clé `INFO` apparaît 4 fois dans chacun des deux journaux `application.log` et `full.log` (ce dernier étant une copie/concaténation du premier).

5. Partie 4 — Organisation des données (brut → clean)

Conformément à la règle énoncée dans le readme (ne jamais modifier les fichiers bruts), une copie de `train.csv` est placée dans `datasets/clean` à l'aide de `cp`, en vue d'un futur nettoyage sans toucher à l'original.

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# cp datasets/brut/train.csv datasets/clean/train.csv
# aucune sortie = copie réussie
```

Résultat / interprétation : Le fichier original reste intact dans `datasets/brut` ; une copie de travail existe désormais dans `datasets/clean`, prête à être nettoyée par le script `preprocess.py`.

6. Partie 5 — Recherche de fichiers par type

La commande `find` parcourt récursivement l'arborescence à la recherche de fichiers correspondant à un motif, ici tous les `.csv` puis tous les `.log`. La commande `ls -ls` liste ensuite le contenu du dossier courant trié par taille décroissante.

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# find . -name "*.csv"
./datasets/clean/train.csv
./datasets/brut/clients.csv
./datasets/brut/train.csv
root@DESKTOP-PTBFF3I:~/IA_Project# find . -name "*.log"
./logs/application.log
./logs/training.log
./logs/loss.log
./logs/full.log
root@DESKTOP-PTBFF3I:~/IA_Project# ls -ls
drwxr-xr-x api backup config datasets documentation logs models scripts shared
```

Résultat / interprétation : Trois fichiers CSV et quatre fichiers de logs sont bien répartis dans les bons dossiers. `ls -ls` confirme la présence des 9 sous-dossiers de premier niveau du projet.

7. Partie 6 — Fusion et filtrage des journaux

L'opérateur `>` permet de rediriger une sortie vers un nouveau fichier : `cat` a servi ici à concaténer deux journaux en un seul fichier `concat.log`, puis `grep` a extrait uniquement les lignes contenant « Loss » du fichier `training.log` pour reconstituer `loss.log`.

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# cat logs/application.log logs/training.log > logs/concat.log
root@DESKTOP-PTBFF3I:~/IA_Project# grep "Loss" logs/training.log > logs/loss.log
root@DESKTOP-PTBFF3I:~/IA_Project# cat logs/loss.log
Epoch 1 : Loss = 0.82
Epoch 2 : Loss = 0.60
Epoch 3 : Loss = 0.45
Epoch 4 : Loss = 0.32
Epoch 5 : Loss = 0.28
```

Résultat / interprétation : Le nouveau fichier `concat.log` regroupe les deux journaux d'origine. Le filtrage par `grep "Loss"` reproduit fidèlement les 5 lignes de pertes déjà observées, prouvant que la commande de redirection fonctionne comme attendu.

8. Partie 7 — Sauvegarde de l'ensemble du projet

La commande `tar -czvf` crée une archive compressée (`c` = create, `z` = gzip, `v` = verbose, `f` = nom de fichier) de tout le dossier `IA_Project`, y compris le dépôt Git, et la stocke dans le dossier `backup/`.

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# find ~/IA_Project -type f | wc -l
63
root@DESKTOP-PTBFF3I:~/IA_Project# tar -czvf backup/projet_IA.tar.gz -C ~ IA_Project
IA_Project/
IA_Project/config/settings.conf
IA_Project/config/model.conf
IA_Project/logs/ ...
IA_Project/.git/ ...
IA_Project/models/modele.txt
IA_Project/datasets/ ...
IA_Project/scripts/preprocess.py
IA_Project/scripts/train.py
IA_Project/backup/projet_IA.tar.gz
```

Résultat / interprétation : Avant sauvegarde, le projet comptait **63 fichiers**. L'archive `backup/projet_IA.tar.gz` a été créée avec succès et regroupe l'ensemble des dossiers (datasets, config, logs, models, scripts, dépôt .git inclus).

9. Partie 8 — Test de restauration de la sauvegarde

Pour vérifier qu'une sauvegarde est réellement exploitable, elle a été extraite dans un dossier de test distinct (`~/restauration_test`) à l'aide de `tar -xzf` (`x` = extract).

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# mkdir -p ~/restauration_test
root@DESKTOP-PTBFF3I:~/IA_Project# tar -xzf backup/projet_IA.tar.gz -C ~/restauration_test
IA_Project/
IA_Project/config/ ...
IA_Project/datasets/ ...
IA_Project/scripts/ ...
IA_Project/.git/ ...
root@DESKTOP-PTBFF3I:~/IA_Project# ls ~/restauration_test
IA_Project
```

Résultat / interprétation : L'extraction restitue intégralement l'arborescence IA_Project dans le dossier de test. La sauvegarde est donc **valide et restaurable**. Un avertissement bénin de tar signalant une date de modification « dans le futur » apparaît pour backup/ (horodatage système du conteneur), sans impact sur l'intégrité des fichiers restaurés.

10. Partie 9 — Installation des outils du projet IA

Enfin, les outils nécessaires au projet (contrôle de version, transfert réseau, supervision des processus, langage Python et gestionnaire de paquets, compilation, décompression) sont installés en une seule commande `apt install`.

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# apt install -y git curl wget htop python3 python3-pip unzip
git is already the newest version (1:2.43.0-1ubuntu7.3).
curl is already the newest version (8.5.0-2ubuntu10.11).
wget is already the newest version (1.21.4-1ubuntu4.4).
python3 is already the newest version (3.12.3-0ubuntu2.1).
The following NEW packages will be installed:
build-essential htop python3-dev python3-pip python3-wheel unzip ...
0 upgraded, 68 newly installed, 0 to remove and 2 not upgraded.
Need to get 80.5 MB of archives.
After this operation, 284 MB of additional disk space will be used.
Fetched 80.5 MB in 1min 5s (1231 kB/s)
Setting up ... Done.
```

Résultat / interprétation : Git, curl et wget étaient déjà présents. **68 nouveaux paquets** ont été installés (dont htop, python3-pip, unzip, et les outils de compilation build-essential nécessaires à certaines bibliothèques Python), pour un total de **284 Mo** d'espace disque supplémentaire. L'environnement dispose désormais de tous les outils requis pour développer, entraîner et déployer le modèle IA du projet.

11. Vérification des outils installés

Une fois l'installation terminée, chaque outil est vérifié individuellement avec son option `--version` (ou `-v`) : cela confirme que le logiciel est correctement installé et accessible depuis n'importe quel dossier.

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project
```

```
root@DESKTOP-PTBFF3I:~/IA_Project# git --version
git version 2.43.0
root@DESKTOP-PTBFF3I:~/IA_Project# curl --version
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 ...
root@DESKTOP-PTBFF3I:~/IA_Project# wget --version
GNU Wget 1.21.4 built on linux-gnu.
root@DESKTOP-PTBFF3I:~/IA_Project# htop --version
htop 3.3.0
root@DESKTOP-PTBFF3I:~/IA_Project# tree --version
tree v2.1.1 © 1996 - 2023 by Steve Baker, Thomas Moore ...
root@DESKTOP-PTBFF3I:~/IA_Project# python3 --version
Python 3.12.3
root@DESKTOP-PTBFF3I:~/IA_Project# pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
root@DESKTOP-PTBFF3I:~/IA_Project# unzip -v
UnZip 6.00 of 20 April 2009, by Debian. Original by Info-ZIP.
```

Résultat / interprétation : Les 8 outils installés (git, curl, wget, htop, tree, python3, pip3, unzip) répondent tous correctement à leur commande de version : l'environnement logiciel est complet et opérationnel pour le projet IA.

12. Récupération d'un jeu de données réel

La commande `wget` a été utilisée pour télécharger un jeu de données public (le célèbre dataset **Iris**) directement depuis GitHub, afin d'avoir un vrai fichier de données exploitable dans `datasets/brut`.

```
■ root@DESKTOP-PTBFF3I: ~/IA_Project/datasets/brut
```



```
root@DESKTOP-PTBFF3I:~/IA_Project# cd datasets/brut
root@DESKTOP-PTBFF3I:~/IA_Project# wget
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
HTTP request sent, awaiting response... 200 OK
Length: 3858 (3.8K) [text/plain]
iris.csv saved [3858/3858]
root@DESKTOP-PTBFF3I:~/IA_Project# cd ~/IA_Project
root@DESKTOP-PTBFF3I:~/IA_Project# ls datasets/brut/
clients.csv iris.csv readme.txt train.csv
root@DESKTOP-PTBFF3I:~/IA_Project# head -5 datasets/brut/iris.csv
sepal_length,sepal_width,petal_length,petal_width,species
5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
```

Résultat / interprétation : Le téléchargement a réussi (code **200 OK**, 3.8 Ko). Le dossier datasets/brut contient désormais un quatrième fichier, iris.csv, avec 5 colonnes numériques et une colonne species — un dataset réel prêt pour des tests de Machine Learning.

13. Gestion des processus

Un petit script Python (`print-pause.py`) qui affiche un compteur toutes les secondes a été créé pour simuler un traitement long (par exemple un entraînement de modèle). Il a été lancé **en arrière-plan** avec `&`, puis surveillé avec `jobs -l` et `ps aux`, avant d'être arrêté.

■ root@DESKTOP-PTBFF3I: ~/IA_Project/scripts

```
root@DESKTOP-PTBFF3I:~/IA_Project# cd ~/IA_Project/scripts
root@DESKTOP-PTBFF3I:~/IA_Project# cat > print-pause.py << 'EOF'
import time

for i in range(1000):
    print(i)
    time.sleep(1)
EOF
root@DESKTOP-PTBFF3I:~/IA_Project# python3 print-pause.py &
[1] 8685
root@DESKTOP-PTBFF3I:~/IA_Project# sleep 2
root@DESKTOP-PTBFF3I:~/IA_Project# jobs -l
[1]+ 8685 Running python3 print-pause.py &
root@DESKTOP-PTBFF3I:~/IA_Project# ps aux | grep print-pause
root 8685 ... python3 print-pause.py
# le compteur défile (0, 1, 2, 3, ... 999) pendant que le processus tourne en tâche de
fond
# Ctrl+C envoyé pour interrompre le job avant la fin des 1000 itérations
^C
[1]+ Done python3 print-pause.py
```

Résultat / interprétation : Le script a bien été lancé en arrière-plan (job **[1]**, PID **8685**), visible à la fois via `jobs -l` (au niveau du shell) et `ps aux` (au niveau du système). L'interruption manuelle (`Ctrl+C`) a stoppé proprement le processus, ce qui démontre la capacité à lancer, surveiller et arrêter un traitement long depuis le terminal.

14. Gestion des utilisateurs et des groupes

Cinq utilisateurs (**alice**, **bob**, **charles**, **diane**, **eva**) représentant une équipe projet ont été créés avec `useradd -m` (option qui crée aussi leur dossier personnel dans `/home`), puis un mot de passe leur a été attribué avec `chpasswd`. Cinq groupes métier (**data**, **models**, **api**, **mlops**, **interns**) ont ensuite été créés avec `groupadd`, et chaque utilisateur a été ajouté au groupe correspondant à son rôle via `usermod -aG`.

■ root@DESKTOP-PTBFF3I: ~/IA_Project/scripts

```

root@DESKTOP-PTBFF3I:~/IA_Project# useradd -m alice
root@DESKTOP-PTBFF3I:~/IA_Project# useradd -m bob
root@DESKTOP-PTBFF3I:~/IA_Project# useradd -m charles
root@DESKTOP-PTBFF3I:~/IA_Project# useradd -m diane
root@DESKTOP-PTBFF3I:~/IA_Project# useradd -m eva
root@DESKTOP-PTBFF3I:~/IA_Project# echo "alice:*****" | chpasswd
root@DESKTOP-PTBFF3I:~/IA_Project# echo "bob:*****" | chpasswd
root@DESKTOP-PTBFF3I:~/IA_Project# echo "charles:*****" | chpasswd
root@DESKTOP-PTBFF3I:~/IA_Project# echo "diane:*****" | chpasswd
root@DESKTOP-PTBFF3I:~/IA_Project# echo "eva:*****" | chpasswd
root@DESKTOP-PTBFF3I:~/IA_Project# ls /home
alice bob charles diane eva
# Création des groupes métier
root@DESKTOP-PTBFF3I:~/IA_Project# groupadd data
root@DESKTOP-PTBFF3I:~/IA_Project# groupadd models
root@DESKTOP-PTBFF3I:~/IA_Project# groupadd api
root@DESKTOP-PTBFF3I:~/IA_Project# groupadd mlops
root@DESKTOP-PTBFF3I:~/IA_Project# groupadd interns
# Affectation de chaque utilisateur à son groupe
root@DESKTOP-PTBFF3I:~/IA_Project# usermod -aG data alice
root@DESKTOP-PTBFF3I:~/IA_Project# usermod -aG data bob
root@DESKTOP-PTBFF3I:~/IA_Project# usermod -aG api charles
root@DESKTOP-PTBFF3I:~/IA_Project# usermod -aG mlops diane
root@DESKTOP-PTBFF3I:~/IA_Project# usermod -aG interns eva
root@DESKTOP-PTBFF3I:~/IA_Project# id alice
uid=1000(alice) gid=1000(alice) groups=1000(alice),1005(data)
root@DESKTOP-PTBFF3I:~/IA_Project# id charles
uid=1002(charles) gid=1002(charles) groups=1002(charles),1007(api)

```

Résultat / interprétation : Les 5 comptes sont créés avec leur dossier personnel, et chacun appartient bien à son groupe métier (alice/bob → data, charles → api, diane → mlops, eva → interns). Un **6^e** utilisateur, **frank**, a ensuite été ajouté de la même façon et rattaché au groupe data, illustrant l'arrivée d'un nouveau membre dans l'équipe.

15. Attribution des propriétaires et des permissions

Chaque dossier du projet a été attribué à un propriétaire et un groupe pertinents avec `chown utilisateur:groupe`, puis des droits d'accès précis ont été fixés avec `chmod` selon la sensibilité du contenu : par exemple 700 pour les sauvegardes (accès strictement réservé au propriétaire) et 775 pour le dossier partagé (accessible en écriture par tout le groupe).

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
# Propriétaires des dossiers
root@DESKTOP-PTBFF3I:~/IA_Project# chown alice:data datasets
root@DESKTOP-PTBFF3I:~/IA_Project# chown alice:data models
root@DESKTOP-PTBFF3I:~/IA_Project# chown charles:api api
root@DESKTOP-PTBFF3I:~/IA_Project# chown diane:mlops logs
root@DESKTOP-PTBFF3I:~/IA_Project# chown diane:mlops backup
root@DESKTOP-PTBFF3I:~/IA_Project# chown bob:data documentation
root@DESKTOP-PTBFF3I:~/IA_Project# chown root:data shared
# Droits d'accès
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 770 datasets
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 750 models
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 770 api
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 740 logs
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 700 backup
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 744 documentation
root@DESKTOP-PTBFF3I:~/IA_Project# chmod 775 shared
root@DESKTOP-PTBFF3I:~/IA_Project# ls -la ~/IA_Project
drwxrwx--- 2 charles api ... api
drwx----- 2 diane mlops ... backup
drwxrwx--- 4 alice data ... datasets
drwxr--r-- 2 bob data ... documentation
drwxr----- 2 diane mlops ... logs
drwxr-x--- 2 alice data ... models
drwxrwxr-x 2 root data ... shared
```

Résultat / interprétation : La sortie de `ls -la` confirme que chaque dossier porte bien le propriétaire, le groupe et les droits attendus : `backup` est totalement privé à `diane` (700), `shared` est ouvert en écriture à tout le groupe `data` (775), et les autres dossiers ont des droits intermédiaires cohérents avec le rôle de chaque équipe (données, modèles, API, logs, documentation).

16. Script d'automatisation `setup_project.sh`

Le livrable central de l'atelier est un script Bash, `setup_project.sh`, qui automatise entièrement la mise en place d'un projet IA : il demande le nom du projet, crée l'arborescence complète, génère un fichier de configuration, installe les logiciels nécessaires, télécharge un dataset réel, crée une archive de sauvegarde, puis affiche un récapitulatif (OK/ECHEC) pour chaque étape.

■ nano ~/setup_project.sh

```
#!/bin/bash
read -p "Entrez le nom du projet : " PROJECT_NAME
BASE_DIR="$HOME/$PROJECT_NAME"
echo "=====
echo "Création du projet : $PROJECT_NAME"
echo "=====

# 1. Arborescence
mkdir -p "$BASE_DIR"/{datasets/brut,datasets/clean,config,logs,scripts,models,api,back
up,documentation,shared}
if [ -d "$BASE_DIR" ]; then ARBO_STATUS="OK"; else ARBO_STATUS="ECHEC"; fi

# 2. Fichier de configuration
cat > "$BASE_DIR/config/settings.conf" << EOF
PROJECT_NAME=$PROJECT_NAME
DATA_PATH=datasets/brut
MODEL_PATH=models
LOG_LEVEL=INFO
API_PORT=8000
AUTHOR=Equipe IA
EOF
if [ -f "$BASE_DIR/config/settings.conf" ]; then CONFIG_STATUS="OK"; else
CONFIG_STATUS="ECHEC"; fi

# 3. Installation des logiciels
echo "Installation des logiciels nécessaires..."
apt update -y > /dev/null 2>&1
apt install -y git curl wget htop tree python3 python3-pip unzip > /dev/null 2>&1
TOOLS="git curl wget htop tree python3 pip3 unzip"
LOGICIELS_STATUS="OK"
for tool in $TOOLS; do
if ! command -v $tool > /dev/null 2>&1; then LOGICIELS_STATUS="ECHEC ($tool
manquant)"; fi
done

# 4. Dataset réel
DATASET_URL="https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
wget -q "$DATASET_URL" -O "$BASE_DIR/datasets/brut/iris.csv"
if [ -s "$BASE_DIR/datasets/brut/iris.csv" ]; then DATASET_STATUS="OK"; else
DATASET_STATUS="ECHEC"; fi

# 5. Sauvegarde
tar -czf "$BASE_DIR/backup/${PROJECT_NAME}.tar.gz" -C "$HOME" "$PROJECT_NAME"
2>/dev/null
ARCHIVE_PATH="backup/${PROJECT_NAME}.tar.gz"

# 6. Récapitulatif
echo "=====
echo "Projet créé"
echo "Nom : $PROJECT_NAME"
echo "Arborescence : $ARBO_STATUS"
echo "Fichier de config : $CONFIG_STATUS"
echo "Logiciels : $LOGICIELS_STATUS"
echo "Datasets : $DATASET_STATUS"
echo "Archive : $ARCHIVE_PATH"
echo "Installation terminée."
echo "=====
```

■ root@DESKTOP-PTBFF3I: ~/test_final

```
root@DESKTOP-PTBFF3I:~/IA_Project# chmod +x setup_project.sh
root@DESKTOP-PTBFF3I:~/IA_Project# ./setup_project.sh
Entrez le nom du projet : IA_Project
=====
Création du projet : IA_Project
=====
Installation des logiciels nécessaires...
=====
Projet créé
Nom : IA_Project
Arborescence : OK
Fichier de config : OK
Logiciels : OK
Datasets : OK
Archive : backup/IA_Project.tar.gz
Installation terminée.
=====
```

Résultat / interprétation : Le script a été testé deux fois avec succès, dans un dossier de test (~ / test_final) puis directement depuis ~ avec un second nom de projet (IA_Project_Final). Les 4 vérifications automatiques (arborescence, configuration, logiciels, dataset) retournent toutes **OK**, ce qui prouve que le script est fiable et reproductible : il permet de recréer un environnement de projet IA complet en une seule commande, avec une seule information à saisir (le nom du projet). Le script final a été copié dans scripts/setup_project.sh du projet, pour être versionné avec le reste du code.

17. Versionnement du projet avec Git et GitHub

Pour finir, l'identité Git a été configurée globalement, puis l'ensemble du projet (scripts, configuration, dataset, script d'automatisation) a été ajouté, commité et poussé vers un dépôt distant sur GitHub, afin d'assurer une sauvegarde externe et un historique des modifications.

■ root@DESKTOP-PTBFF3I: ~/IA_Project

```
root@DESKTOP-PTBFF3I:~/IA_Project# git config --global user.name "..."  
root@DESKTOP-PTBFF3I:~/IA_Project# git config --global user.email "..."  
root@DESKTOP-PTBFF3I:~/IA_Project# git config --global --list  
user.name=...  
user.email=...  
credential.helper=store  
root@DESKTOP-PTBFF3I:~/IA_Project# git init  
Reinitialized existing Git repository in /root/IA_Project/.git/  
root@DESKTOP-PTBFF3I:~/IA_Project# git remote -v  
origin https://github.com/.../Atelier-Linux-IA.git (fetch)  
origin https://github.com/.../Atelier-Linux-IA.git (push)  
root@DESKTOP-PTBFF3I:~/IA_Project# git add .  
root@DESKTOP-PTBFF3I:~/IA_Project# git commit -m "Ajout du script setup_project.sh et du travail complet"  
[main 73a3933] Ajout du script setup_project.sh et du travail complet  
6 files changed, 208 insertions(+)  
create mode 100644 backup/IA_Project.tar.gz  
create mode 100644 datasets/brut/iris.csv  
create mode 100644 logs/concat.log  
create mode 100644 scripts/print-pause.py  
create mode 100755 scripts/setup_project.sh  
root@DESKTOP-PTBFF3I:~/IA_Project# git branch -M main  
root@DESKTOP-PTBFF3I:~/IA_Project# git push -u origin main  
Enumerating objects: 17, done.  
Writing objects: 100% (12/12), 58.00 KiB | 5.27 MiB/s, done.  
1a5e2c5..73a3933 main -> main  
branch 'main' set up to track 'origin/main'.
```

Résultat / interprétation : Le commit 73a3933 a bien été créé (208 lignes ajoutées, 6 fichiers incluant le script `setup_project.sh`, le dataset `iris.csv` et les archives de sauvegarde), puis poussé avec succès vers le dépôt distant `Atelier-Linux-IA` sur GitHub. Le projet est désormais versionné et accessible en ligne, avec un historique de commits traçant chaque étape de l'atelier.

18. Conclusion

Cet atelier a permis de mettre en place, de bout en bout, un environnement Linux complet et opérationnel pour un projet de Machine Learning : structuration du projet avec une arborescence claire, exploration et manipulation des fichiers de données et de logs, séparation stricte entre données brutes et données nettoyées, sauvegarde suivie d'un test de restauration réussi, installation et vérification des outils logiciels indispensables, récupération d'un dataset réel, gestion des processus en arrière-plan, création d'utilisateurs/groupes avec des permissions différenciées par rôle, automatisation complète via le script `setup_project.sh`, et enfin versionnement du projet sur GitHub. L'ensemble des commandes a été exécuté avec succès dans un sous-système Linux WSL2 (Ubuntu 24.04).